

PROMETHEUS v8.2 — THE BODY WIRING SPRINT

From Manifest to MP4: Closing the Product Loop

Version: 8.2 | **Status:** LOCKED | **Date:** 2026-06-21
Sprint Duration: 14 Days | **Target:** Upload → Joseph Edit → MP4 (1080×1920)
Stack: R3F + Remotion + FFmpeg | **Render Mode:** Sequential Local CLI
Governor: Judgment Layer (already implemented) | **Variation:** Candidate Genomes + Replay Ledger (already implemented)

1. EXECUTIVE SUMMARY

Current State: The brain is alive. The backend generates deterministic, judgment-backed, variation-aware Joseph edit manifests. 18 backend tests pass. Determinism and variation are mathematically proven at the manifest level.

The Gap: The body cannot execute the brain’s decisions. The upload route does not call the orchestrator. The render pipeline emits 1920×1080 instead of 1080×1920. Source videos 404 in Chromium because `file:///` is blocked. SFX files are zero-byte stubs. Font selection is hard-coded to one fallback. Music analysis is a skeleton with TODOs. No test has ever proven a non-black pixel in a final MP4.

The Goal: Wire the brain to the body. Make `upload video → orchestrate → render → MP4` a single provable command. Every ticket in this roadmap is a bridge between the manifest world and the pixel world.

2. THE BRAIN VS. BODY GAP (DIAGNOSTIC FINDINGS)

#	Finding	Severity	File / Line
1	Upload route does not call Joseph Orchestrator	Existential	backend/src/upload-routes.ts:105
2	Render-jobs builds old manifest shape, not UnifiedRenderManifest	Existential	backend/src/render-jobs/routes.ts:99
3	Worker sample output is 1920×1080, not 1080×1920	Systemic	apps/worker/src/index.ts (implied)
4	Source video 404s in Chromium (<code>file:///</code> blocked)	Systemic	Worker render logs
5	SFX files are 40 zero-byte stubs	Systemic	remotion-app/public/sfx/

Table 1 – continued

#	Finding	Severity	File / Line
6	SFX cue resolution expects <code>\${cue}.mp3</code>, stubs are <code>\${cue}_1.mp3</code>... <code>\${cue}_5.mp3</code>	Systemic	backend/src/audio/mix-audio.ts:53
7	Only 22 local fonts; hard-coded Antenna fallback	Isolated	remotion-app/public/fonts/library/
8	Music analysis is skeleton (librosa/essentia TODOs)	Isolated	backend/src/music/README.md:8
9	No full-render pixel test exists	Existential	N/A —missing entirely
10	T01–T09 render infrastructure marked <code>NOT_STARTED</code> in tracker	Systemic	PROMETHEUS_BUILD.md

3. THE ROADMAP: 14-DAY SPRINT

Week 1: Manifest → Render Contract (Days 1–5)

Day	Ticket	Agent	Files	Deliverable	Gate
1	T33: Asset Contract	Any	packages/ shared-types/ src/asset- resolver.ts, backend/src/ asset/ local-asset- resolver.ts	Browser-safe URL File-path dual resolution	Typecheck + unit test
1	T34: Upload → Orchestrator Wiring	Any	backend/src/ upload- routes.ts, backend/src/ render-jobs/ routes.ts	Upload completion triggers <code>orchestrateRender()</code>	Integration test
2	T35: Manifest Unification	Any	backend/src/ render-jobs/ routes.ts, apps/worker/ src/index.ts	Worker receives UnifiedRenderManifest, not legacy shape	Worker typecheck

Table 2 – continued

Day	Ticket	Agent	Files	Deliverable	Gate
3	T36: Vertical Resolution Enforcement	Any	remotion-app/ src/joseph- bundle.ts, apps/worker/ src/index.ts	Render output is 1080×1920, provable via ffmpeg	ffmpeg assertion
4	T37: Browser-Safe Video Serving	Any	backend/src/ asset/video- server.ts, worker asset resolver	Uploaded video served via http://localhost:PORT/ or copied to public/	Render log: no 404
5	T38: Integration Test –Manifest to Silent MP4	Any	scripts/test- full-render.ts	Silent MP4 rendered from orchestrator manifest, 100KB, 1080×1920	File exists + ffmpeg

Week 2: Media Pipeline Hardening (Days 6–10)

Day	Ticket	Agent	Files	Deliverable	Gate
6	T39: SFX Real Files + Seeded Selection	Any	remotion-app/ public/sfx/, backend/src/ audio/ mix-audio.ts, backend/src/ director/ joseph- director.ts	40 real MP3s (or 8 base + 5 variants); director seeds _1... _5 selection	Audio mix test passes
7	T40: Font Pipeline –5 Hero Fonts	Any	remotion-app/ public/fonts/ hero/, backend/ src/director/ joseph- director.ts, remotion-app/ src/ compositions/ JosephEdit.tsx	5 WOFF2 hero fonts loaded via @font-face; director seeds font selection	Font render test
8	T41: Music Beat-Match Integration	Any	backend/src/ music/, backend/src/ audio/ mix-audio.ts	Librosa/Essentia beat grid → loop at phrase boundaries, fade at video end	Audio mix test

Table 3 – continued

Day	Ticket	Agent	Files	Deliverable	Gate
9	T42: Audio Mix End-to-End	Any	scripts/test-full-render.ts	Final MP4 has audio stream, LUFS -14, music ducks under voice	ffprobe + ebur128
10	T43: Render Entry Contract Verification	Any	remotion-app/src/joseph-bundle.ts, remotion-app/src/joseph-entry.tsx	Bundle entry only registers Joseph; no unrelated preview code	Bundle time < 30s

Week 3: Pixel Proof & Production (Days 11–14)

Day	Ticket	Agent	Files	Deliverable	Gate
11	T44: Full Render Pixel Test	Any	scripts/test-full-render.ts, scripts/assert-nonblack.ts	Extract frame 100 as PNG; assert > 5% non-black pixels	Image hash diff
12	T45: Source Video Visibility Test	Any	scripts/test-full-render.ts	Extract frame; assert source video pixels are present (not pure text overlay)	Perceptual hash
13	T46: Production Upload → MP4 Loop	Any	backend/src/upload-routes.ts, apps/worker/src/index.ts	Full user journey: POST video → receive MP4 URL	E2E test
14	T47: CI Render Gate + Tracker Cleanup	Any	.github/workflows/render-gate.yml, PROMETHEUS_BUILD.md	CI runs full render test on push; tracker reflects true state	CI green

4. DETAILED TICKET SPECIFICATIONS**T33: Asset Contract**

Problem: Remotion/R3F in Chromium cannot load `file:///C:/Users/.../uploads/video.mp4`. FFmpeg cannot load `http://localhost:3000/video.mp4` if the server is down. The manifest needs both representations.

Files:

- Create `packages/shared-types/src/asset-resolver.ts`

- Create backend/src/asset/local-asset-resolver.ts

Interface:

```
export interface MediaReference {
  assetId: string;      // e.g., "asset://source_video_1"
  browserUrl: string;   // e.g., "/uploads/abc123.mp4" (served by local HTTP or copied to public/)
  filePath: string;     // e.g., "C:/.../uploads/abc123.mp4" (for FFmpeg)
}

export interface AssetResolver {
  resolveBrowser(assetId: string): string;
  resolveFile(assetId: string): string;
  registerUpload(sourcePath: string, jobId: string): MediaReference;
}

export class LocalAssetResolver implements AssetResolver {
  constructor(private publicDir: string, private uploadDir: string, private port?: number) {}
  // If port provided, serve via http://localhost:${port}/uploads/${jobId}/${filename}
  // Else copy symlink into publicDir and return relative path
}
```

Rules:

- registerUpload copies or symlinks the uploaded file into a browser-accessible location
- browserUrl must never start with file:/// or C:/
- filePath must be absolute for FFmpeg
- If port is provided, start a minimal http.createServer on that port serving uploadDir statically
- If port is not provided, copy file into publicDir and return /uploads/\${jobId}/\${filename}

Test: packages/shared-types/test/asset-resolver.test.ts

1. registerUpload returns both browserUrl and filePath
2. browserUrl does not contain file:/// or C:/
3. filePath is absolute
4. resolveBrowser throws for unknown assetId
5. HTTP server mode serves file at browserUrl (200 OK)

Fallback: If HTTP server fails to bind, fall back to copy-to-publicDir.

T34: Upload → Orchestrator Wiring

Problem: backend/src/upload-routes.ts:105 creates an edit session and completes the upload, but never calls orchestrateRender().

Files:

- Edit backend/src/upload-routes.ts
- Edit backend/src/render-jobs/routes.ts

Implementation: In the upload completion handler (where `editSession` is finalized):

```
import { orchestrateRender } from "../../director/orchestrator";
import { ReplayLedger } from "../../ledger/replay-ledger";
import { PromptRegistry } from "../../director/prompt-governance";
import { LocalAssetResolver } from "../../asset/local-asset-resolver";

// After upload is saved to disk:
const resolver = new LocalAssetResolver(PUBLIC_DIR, UPLOAD_DIR, 3456);
const mediaRef = resolver.registerUpload(savedPath, editSession.id);

const ledger = new ReplayLedger();
const registry = new PromptRegistry();

const result = await orchestrateRender({
  sourceVideoPath: mediaRef.filePath,
  transcriptPath: transcriptPath, // from Whisper output
  audioPath: mediaRef.filePath, // source video audio
  musicPath: editSession.musicPath, // optional, from user selection
  promptText: editSession.promptText,
  profile: editSession.profile ?? "joseph_cinematic",
  uploadInstanceId: editSession.id,
  retryIndex: 0,
}, ledger, registry);

// Store result.manifest in the render job
await createRenderJob({
  editId: editSession.id,
  manifest: result.manifest,
  variationKey: result.variationKey,
  evidencePath: result.evidence, // evidence-preservation writes this
  status: "pending",
});
```

Rules:

- Do NOT break the existing upload flow for non-Joseph edits
- Only trigger orchestrator if `editSession.type === "joseph"` or `editSession.profile` is set
- If orchestrator throws, mark job as failed with error message, do not crash the upload handler
- The manifest stored in the render job must be a valid `UnifiedRenderManifest` (Zod-validated)

Test: `backend/src/upload-routes.test.ts` (integration)

1. POST upload completion with profile: "joseph_aggressive" → creates render job with `UnifiedRenderManifest`
2. Manifest has width: 1080, height: 1920
3. Render job status is pending
4. Evidence directory exists at `~/.prometheus/evidence/{jobId}/`

T35: Manifest Unification

Problem: backend/src/render-jobs/routes.ts:99 builds a legacy manifest shape. The worker expects UnifiedRenderManifest.

Files:

- Edit backend/src/render-jobs/routes.ts
- Verify apps/worker/src/index.ts imports UnifiedRenderManifest from @prometheus/shared-types

Implementation: The render job creation route must accept and store UnifiedRenderManifest directly. No translation layer.

```
import { UnifiedRenderManifest, UnifiedRenderManifestSchema } from "@prometheus/shared-types";

// In render job creation:
const manifest = UnifiedRenderManifestSchema.parse(req.body.manifest);
// Store manifest JSONB in DB
```

The worker's renderFromManifest(manifest: UnifiedRenderManifest) must receive the exact schema.

Rules:

- Delete any legacy manifest interfaces in render-jobs/routes.ts
- Worker must validate manifest with Zod at entry before passing to Remotion
- If validation fails, log failure tags and mark job failed

Test: apps/worker/test/manifest-validation.test.ts

1. Valid UnifiedRenderManifest passes Zod validation
2. Legacy manifest (missing profile or cuts in wrong shape) fails with clear error
3. Worker calls renderMedia with inputProps: { manifest, frameData }

T36: Vertical Resolution Enforcement

Problem: Worker sample output was 1920×1080. The manifest says 1080×1920 but the render pipeline ignores it.

Files:

- Verify remotion-app/src/joseph-bundle.ts
- Verify remotion-app/src/joseph-entry.tsx
- Edit apps/worker/src/index.ts if needed

Implementation: In joseph-entry.tsx:

```
<Composition
  id="JosephEdit"
  component={JosephEdit}
  durationInFrames={2700} // 90s @ 30fps
  fps={30}
  width={1080}
```

```
height={1920}
defaultProps={{ manifest: null, frameData: null }}
</>
```

In the worker, when calling `renderMedia`:

```
await renderMedia({
  composition,
  serveUrl,
  codec: "h264",
  crf: 18,
  outputLocation,
  inputProps: { manifest, frameData },
  // Do NOT override width/height here — let Composition own it
});
```

Rules:

- Composition width and height are hardcoded to 1080 and 1920, never read from manifest (manifest is editorial, not structural)
- VideoPlane component must center-crop the source video to 9:16 aspect ratio (v8.1 spec page 12, FT-027)
- If source video is horizontal (16:9), crop center: `crop=ih*9/16:ih` then scale to 1080:1920

Test: `scripts/test-vertical-render.ts`

```
import { execSync } from "child_process";
const info = execSync(`ffprobe -v error -select_streams v:0 -show_entries stream=width,height -of
↪ csv=s=x:p=0 ${mp4Path}`).toString().trim();
console.assert(info === "1080x1920", `Expected 1080x1920, got ${info}`);
```

Fallback: If `ffprobe` is not installed, skip the assertion and log a warning.

T37: SFX Real Files + Seeded Selection

Problem: 40 zero-byte stubs. Director emits `whoosh_fast` but mixer looks for `whoosh_fast.mp3`, while stubs are `whoosh_fast_1.mp3`.

Files:

- `remotion-app/public/sfx/` (generate or acquire)
- `backend/src/director/joseph-director.ts`
- `backend/src/audio/mix-audio.ts`

Implementation:

Option A (Preferred): Generate real 0.5s MP3s with FFmpeg:

```
for cue in whoosh_fast whoosh_slow impact_deep impact_sharp riser_short sub_drop glitch_digital
↪ pop_text; do
  for i in 1 2 3 4 5; do
```



```
ffmpeg -f lavfi -i anullsrc=r=44100:cl=mono -t 0.5 -acodec libmp3lame -q:a 9 "remotion-app/public/
  ↳ sfx/${cue}_${i}.mp3"
done
done
```

If FFmpeg is unavailable, acquire 8 real SFX MP3s from a free library (freesound.org, mixkit.co) and duplicate them 5× with `_1..._5` suffixes.

Director Seeded Selection: In `joseph-director.ts`, when assigning SFX to a cut or text event:

```
const variant = Math.floor(seededRandom(seed + sfxIndex) * 5) + 1; // 1..5
const cueName = `${cue}_${variant}`; // e.g., "whoosh_fast_3"
```

Mixer Resolution: In `mix-audio.ts`, `resolveSfxPath` already looks for `${cue}.mp3`. It must now accept the `_N` suffix:

```
const resolveSfxPath = (sfxAssetsDir: string, cue: string): string => {
  const candidate = path.join(sfxAssetsDir, `${cue}.mp3`);
  // cue already includes _N variant from director
  if (!fs.existsSync(candidate)) {
    throw new Error(`SFX asset missing: ${candidate}`);
  }
  return candidate;
};
```

Rules:

- All 40 files must be > 0 bytes (real audio data)
- Director must use `seededRandom` for variant selection, not `Math.random`
- If a variant is missing, fall back to `_1` (base variant)

Test: `backend/src/audio/mix-audio.test.ts` (extend existing)

1. `resolveSfxPath("whoosh_fast_3")` returns valid path
2. `resolveSfxPath("nonexistent")` throws
3. Director generates 6 candidates; all have SFX cues with `_1..._5` variants
4. Same seed = same SFX variant selection (determinism)

T38: Font Pipeline — 5 Hero Fonts

Problem: Only 22 fonts locally. JosephEdit hard-codes Antenna. No seeded selection.

Files:

- `remotion-app/public/fonts/hero/` (create, populate)
- `remotion-app/src/compositions/JosephEdit.tsx`
- `backend/src/director/joseph-director.ts`

Implementation: Select 5 hero fonts that cover the Joseph aesthetic:

1. **Antenna** (existing, bold geometric sans)
2. **Bebas Neue** (tall, aggressive)
3. **Montserrat Bold** (clean, cinematic)
4. **Oswald Bold** (condensed, minimal)
5. **Inter Bold** (neutral, readable)

Download WOFF2 subsets for each (or full WOFF2 if subsetting is deferred).

In `remotion-app/public/fonts/hero/fonts.css`:

```
@font-face { font-family: 'HeroAntenna'; src: url('/fonts/hero/antenna-bold.woff2') format('woff2');
  ↳ font-weight: 700; }
@font-face { font-family: 'HeroBebas'; src: url('/fonts/hero/bebas-neue.woff2') format('woff2');
  ↳ font-weight: 400; }
/* ... etc ... */
```

In `JosephEdit.tsx`, import the CSS and use `fontFamily` from manifest:

```
import "../../public/fonts/hero/fonts.css";
// In KineticText:
fontFamily={textOverlay.fontFamily} // from manifest
```

In `joseph-director.ts`, seeded font selection per text overlay:

```
const heroFonts = ["HeroAntenna", "HeroBebas", "HeroMontserrat", "HeroOswald", "HeroInter"];
const fontFamily = seededPick(seed + wordIndex, heroFonts);
```

Rules:

- DOM Canvas is PRIMARY (v8.1 spec page 36). `opentype.js` is stretch goal only.
- If a font fails to load in R3F, fallback to `HeroInter` (most readable)
- Font files must be < 100KB each (WOFF2 subsetting to Latin + numbers)

Test: `backend/src/director/joseph-director.test.ts` (extend)

1. Generated manifest text overlays have `fontFamily` field
2. `fontFamily` is one of the 5 hero fonts
3. Same seed = same font selection

Fallback: If WOFF2 files cannot be acquired, use system font stack with CSS `font-family: 'HeroAntenna', 'Arial Black', sans-serif`; and skip the seeded selection for MVP.

T39: Music Beat-Match Integration

Problem: Music module is a skeleton. No beat grid analysis. No phrase boundary looping. No fade at video end.

Files:

- `backend/src/music/analyze.ts` (create or extend)

- backend/src/audio/mix-audio.ts
- backend/src/director/joseph-director.ts

Implementation: If librosa/essentia are not available in the Node.js environment, use a **pure-FFmpeg** beat detection as MVP:

```
export function detectBeatGrid(audioPath: string): { beats: number[]; tempo: number; durationMs: number } {
  // Use FFmpeg aubdetect or ebur128 silence detection as proxy
  const result = execSync(`ffmpeg -i "${audioPath}" -af "ebur128=peak=true" -f null - 2>&1`).toString();
  // Parse peaks as proxy beats
  // Fallback: evenly spaced beats at 128 BPM if detection fails
  const durationSec = getDuration(audioPath);
  const bpm = 128;
  const beatInterval = 60 / bpm;
  const beats = Array.from({ length: Math.floor(durationSec / beatInterval) }, (_, i) => i *
    beatInterval * 1000);
  return { beats, tempo: bpm, durationMs: durationSec * 1000 };
}
```

In the director, when a music track is provided:

1. Analyze beat grid
2. If music duration < video duration, loop at the last detected phrase boundary (or every 4 beats)
3. If music duration > video duration, fade out at videoDurationMs - 500ms

In mix-audio.ts, the music track must be pre-trimmed/looped before mixing:

```
// Pre-process music to match video duration
const trimmedMusic = path.join(tmpDir, "music-trimmed.mp3");
execSync(`ffmpeg -y -i "${musicPath}" -af "afade=t=out:st=${(videoDurationMs - 500) / 1000}:d=0.5" -t
  ${videoDurationMs / 1000} -c:a libmp3lame -q:a 2 "${trimmedMusic}"`);
// Use trimmedMusic in filter_complex instead of original
```

Rules:

- Music is optional. If no musicPath, skip music mixing entirely (voice + SFX only)
- If beat detection fails, use 128 BPM constant as fallback
- Fade must be 0.5s linear fade-out
- Loop must be seamless (crossfade 0.1s at loop point if possible)

Test: backend/src/music/analyze.test.ts

1. detectBeatGrid returns array of beat timestamps in ms
2. Beats are monotonically increasing
3. Fallback 128 BPM produces ~3.75 beats/second

T40: Browser-Safe Video Serving

Problem: file:/// URLs are blocked in Chromium. The source video must be served via HTTP or copied to the public directory.

Files:

- backend/src/asset/video-server.ts (create)
- apps/worker/src/index.ts
- backend/src/upload-routes.ts

Implementation: Two modes, selected by environment variable VIDEO_SERVE_MODE:

Mode A: HTTP Server (preferred for large files)

```
// video-server.ts
import http from "http";
import fs from "fs";
import path from "path";

export function createVideoServer(uploadDir: string, port: number): http.Server {
  return http.createServer((req, res) => {
    const filePath = path.join(uploadDir, req.url?.replace("/uploads/", "") ?? "");
    // Validate filePath is within uploadDir (security)
    if (!filePath.startsWith(uploadDir)) {
      res.writeHead(403); res.end(); return;
    }
    const stream = fs.createReadStream(filePath);
    res.writeHead(200, { "Content-Type": "video/mp4" });
    stream.pipe(res);
  }).listen(port);
}
```

Mode B: Copy to Public (simpler, duplicates storage)

```
fs.copyFileSync(sourcePath, path.join(publicDir, "uploads", jobId, filename));
// browserUrl: "/uploads/${jobId}/${filename}"
```

In the worker, before rendering:

```
// Ensure the video server is running or file is in public/
const browserUrl = assetResolver.resolveBrowser(manifest.sourceVideoAssetId);
// Pass browserUrl to Remotion inputProps
```

Rules:

- Default to Mode B (copy to public) for MVP simplicity
- Mode A is a config flag for Phase 2
- The VideoPlane component in R3F must use the browserUrl, not filePath
- If browserUrl 404s, render must fail fast with clear error (not render black frames)

Test: backend/src/asset/video-server.test.ts

1. Mode B: resolveBrowser returns path starting with /uploads/
2. File exists in publicDir after registerUpload
3. Mode A: HTTP request to browserUrl returns 200 and video/mp4 headers

T41–T46: Full Render Integration Tests

File: scripts/test-full-render.ts

Implementation: A single script that runs the entire loop:

```
#!/usr/bin/env tsx
import { orchestrateRender } from "../backend/src/director/orchestrator";
import { renderFromManifest } from "../apps/worker/src/index";
import { mixAudio } from "../backend/src/audio/mix-audio";
import { execSync } from "child_process";
import fs from "fs";

async function main() {
  // 1. Orchestrate
  const result = await orchestrateRender({ /* mock or real inputs */ });

  // 2. Mix audio
  const mixedAudio = await mixAudio({
    voiceAudioPath: result.manifest.sourceAudioPath,
    musicAudioPath: result.manifest.musicAudioPath,
    sfxAssetsDir: "../remotion-app/public/sfx",
    manifest: result.manifest,
  });

  // 3. Render silent video
  const silentVideo = await renderFromManifest(result.manifest);

  // 4. Mux
  const finalMp4 = `./output/test-${Date.now()}.mp4`;
  execSync(`ffmpeg -y -i "${silentVideo}" -i "${mixedAudio}" -c:v copy -c:a aac -b:a 192k -ar 48000
  ↪ -shortest "${finalMp4}"`);

  // 5. Assert
  const probe = execSync(`ffprobe -v error -show_entries format=duration,bit_rate -show_entries
  ↪ stream=width,height,codec_name -of json "${finalMp4}"`).toString();
  const info = JSON.parse(probe);

  console.assert(info.streams.some((s: any) => s.width === 1080 && s.height === 1920), "Vertical
  ↪ resolution");
  console.assert(info.format.duration <= 90, "90s cap");
  console.assert(info.streams.some((s: any) => s.codec_name === "h264"), "H.264 video");
  console.assert(info.streams.some((s: any) => s.codec_name === "aac"), "AAC audio");
  console.assert(fs.statSync(finalMp4).size > 100 * 1024, "File size > 100KB");

  // 6. Non-black frame
```

```

execSync(`ffmpeg -y -i "${finalMp4}" -ss 00:00:03 -vframes 1 ./output/test-frame.png`);
const frameData = fs.readFileSync("./output/test-frame.png");
// Simple check: PNG of pure black is ~3KB. Real frame is > 10KB.
console.assert(frameData.length > 10000, "Frame has visual content (not pure black)");

console.log("FULL RENDER PASS: ALL ASSERTIONS GREEN");
}

main().catch(console.error);

```

Rules:

- Script must run in < 10 minutes (GPU) or < 15 minutes (software)
- If ffprobe or ffmpeg is missing, skip physical assertions and log warnings
- Must use a real (or mock) video file of at least 10 seconds duration
- Must clean up temp files after run

T47: CI Render Gate + Tracker Cleanup

File: .github/workflows/render-gate.yml

Implementation:

```

name: Prometheus Render Gate
on: [push, pull_request]
jobs:
  full-render:
    runs-on: ubuntu-latest
    timeout-minutes: 20
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: sudo apt-get update && sudo apt-get install -y ffmpeg
      - run: npm ci
      - run: npm run build --workspace=packages/shared-types
      - run: npm run build --workspace=backend
      - run: npm run build --workspace=apps/worker
      - run: npm run build --workspace=remotion-app
      - run: npx tsx scripts/test-full-render.ts
      - name: Verify output
        run: |
          ls -lh output/*.mp4
          ffprobe -v error -select_streams v:0 -show_entries stream=width,height -of csv=s=x:p=0 output/
            ↪ *.mp4 | grep "1080x1920"

```

Tracker Cleanup: Update PROMETHEUS_BUILD.md:

- Mark T01–T09 as READY_FOR_REVIEW (or DONE)

- Mark T33–T47 as `IN_PROGRESS` / `READY_FOR_REVIEW`
- Remove stale blockers that are no longer true
- Add T48+ for Phase 2 hooks (cloud, distributed, Demucs, etc.)

5. RISK REGISTER

#	Risk	Likelihood	Impact	Mitigation
1	FFmpeg not installed on user machine	High	Existential	Bundle FFMpeg static binary or require install
2	Chromium software render too slow (> 15 min)	Medium	Systemic	Accept for MVP; GPU probe in Phase 2
3	Source video codec incompatible with Remotion	Medium	Systemic	Pre-transcode to H.264 before render
4	Font WOFF2 files cannot be acquired	Low	Isolated	System font stack fallback
5	Real SFX library licensing issues	Low	Isolated	Generate synthetic tones with FFMpeg
6	Upload route refactor breaks existing non-Joseph edits	Medium	Systemic	Feature-flag: only run orchestrator if <code>profile</code> is set
7	Full render test flaky in CI (timeouts, Chrome zombies)	Medium	Systemic	20-min timeout, SIGKILL cleanup, retry once
8	Memory pressure: 523MB Chrome + video texture + FFMpeg	Medium	Systemic	90s cap keeps it bounded; warn if < 4GB RAM

6. GO/NO-GO GATES

Gate	Day	Check	Pass Criteria	Blocker Action
G1	1	Asset Contract	<code>browserUrl</code> has no <code>file:///</code>	Fix resolver
G2	3	Manifest Unification	Worker validates Zod without error	Fix schema drift
G3	5	Silent MP4	File > 100KB, 1080×1920 via <code>ffprobe</code>	Fix vertical crop / composition
G4	7	SFX Real	All 40 files > 0 bytes, <code>mix-audio</code> test passes	Acquire or generate real audio
G5	10	Audio Mix	MP4 has AAC stream, LUFS -14	Fix <code>filter_complex</code>
G6	12	Pixel Proof	Frame PNG > 10KB, non-black	Fix source video visibility
G7	14	CI Green	<code>render-gate.yml</code> passes on push	Stabilize flaky tests

7. WHAT “DONE” LOOKS LIKE

A user can:

1. Upload a video via the API
2. Select “Joseph Aggressive” (or cinematic/minimal)
3. Wait 5–10 minutes
4. Receive a 1080×1920 vertical MP4 with:
 - Cuts on audio beats
 - Kinetic text overlays in the top 40% of frame
 - SFX synced to cuts and high-energy words
 - Music that ducks under voice
 - Camera moves (push-in, dutch, shake) per profile
 - No text-on-face overlap (heuristic, < 5% frames)
 - Deterministic: re-upload with same key = identical output
 - Varied: new upload = different (but equally high-quality) edit

And the system preserves:

- Every candidate manifest in evidence
- Every judgment verdict in the Replay Ledger
- Every failure tag for future IRL training

8. SIGN-OFF

This roadmap is **additive only**. It does not rewrite the Judgment Layer, Orchestrator, Sequence Memory, or any T10–T30 work. It bridges the manifest world to the pixel world.

Approved by: Human Architect (Final Authority)

Date: 2026-06-21

Next Action: Execute T33 (Asset Contract). Begin 14-day sprint.